

Proton Energy-Momentum Tensor Measurement

A Detailed Physics Note for the PyQUDA Implementation

July 15, 2026

Abstract

This note documents the complete set of formulas, conventions, and algorithmic steps used in the PyQUDA-based measurement of the proton matrix elements of the energy-momentum tensor (EMT). We cover the proton interpolating field, the connected two-point function, the fixed-sink sequential-source method for connected three-point functions, the covariant derivative insertion for the quark EMT, gradient flow, and the output data structures. The note is written for lattice QCD practitioners and aims to be sufficiently precise that every einsum in the code can be traced back to a well-defined mathematical expression. Throughout, we adopt the Euclidean metric with Hermitian gamma matrices and the DeGrand–Rossi chiral basis as implemented in QUDA and exposed through PyQUDA.

Contents

1	Conventions and Gamma Matrix Setup	1
1.1	Euclidean Gamma Matrices	1
1.2	PyQUDA Gamma Indexing	1
1.3	Charge Conjugation Matrix	2
1.4	Propagator Data Layout	2
1.5	Momentum Phases	2
I	Proton Interpolating Field and Two-Point Function	2
2	Proton Interpolating Field	3
2.1	Supported Interpolators	3
2.2	Epsilon Tensor Convention	3
3	Spin and Parity Projection	4
3.1	Positive Parity Projector	4
3.2	Spin Projectors	4
3.3	Combined Parity-Spin Projectors	4
4	Proton Two-Point Function	5
4.1	Definition	5
4.2	Wick Contraction Result	5
4.3	Sign Convention	5
4.4	Physical Interpretation of the Two Terms	5
5	Optimized Two-Point Contraction	6
5.1	Term 1: Step-by-Step Decomposition	6
5.2	Term 2: Step-by-Step Decomposition	7
5.3	Memory Optimization Rationale	8

II	Proton Sequential Source and Connected Three-Point Function	8
6	Connected Proton Three-Point Function	8
6.1	Definition before the Sequential Trick	8
6.2	Sequential-Source Strategy	9
7	Proton Fixed-Sink Sequential Source	9
7.1	Flavor 1: Up-Quark Insertion	9
7.2	Flavor 2: Down-Quark Insertion	10
7.3	From Diquark Contraction to Sequential Inversion	11
7.4	Sequential Object Shape	12
III	Proton EMT Contractions	12
8	Scalar Diagnostic: C_3^χ	12
9	Connected Quark EMT Insertion	12
9.1	The Quark EMT Operator	12
9.2	First Term: Derivative on the Forward Line	13
9.3	Second Term: Derivative on the Sequential Line	13
9.4	Full Connected Quark EMT	14
10	Key Differences from the Meson EMT	14
11	Gradient Flow	15
11.1	Flow Schedule	15
11.2	Flowing Fermion and Gauge Fields Together	15
11.3	Physical Significance	15
12	Code Structure Overview	16
13	Output Structure	16
13.1	HDF5 Output for Connected Proton Three-Point	16
13.2	HDF5 Output for Disconnected Pieces	18
13.3	File Organization	18
14	Disconnected Analysis	18
14.1	Combined Proton EMT	18
14.2	Disconnected Quark Contribution	18
14.3	Disconnected Gluon Contribution	19
14.4	Ringed-Fermion Normalization	19
IV	Appendices	20
A	Summary of Einsum Index Conventions	20
B	Gamma Matrix Reference	21
C	Complete Two-Point Contraction Einsum Map	21
D	List of Parameters	22

E	Sign Convention Summary	23
F	Validation and Regression Evidence	23
F.1	Validated Test Setup	24
F.2	Raw-Only Sequential Builder Regression	24
F.3	Gauge Covariance of the Connected Three-Point Function	24
F.4	Gradient-Flow Covariance	25
F.5	Momentum Phase Checks	25
F.6	Up-Quark Insertion Term Sanity	25

1 Conventions and Gamma Matrix Setup

1.1 Euclidean Gamma Matrices

We work in four-dimensional Euclidean spacetime with coordinates x_μ , $\mu = 1, 2, 3, 4$. The spatial directions are 1, 2, 3 and the temporal direction is $\mu = 4$. The Euclidean gamma matrices satisfy

$$\{\gamma_\mu, \gamma_\nu\} = 2\delta_{\mu\nu}\mathbb{K}_{4\times 4}, \quad \gamma_\mu^\dagger = \gamma_\mu. \quad (1)$$

The chiral matrix is

$$\gamma_5 \equiv \gamma_1\gamma_2\gamma_3\gamma_4, \quad \gamma_5^\dagger = \gamma_5, \quad \gamma_5^2 = \mathbb{K}. \quad (2)$$

1.2 PyQUDA Gamma Indexing

PyQUDA uses the QUDA bitmask convention for gamma matrices. The 16 independent bilinear structures are ordered as

$$\text{my_gammas} = ["5", "T", "T5", "X", "X5", "Y", "Y5", "Z", "Z5", "I", "SXT", "SXY", "SXZ", "SYT", "SYZ", "SZT"]. \quad (3)$$

The corresponding QUDA-internal integer identifiers are

$$\text{pyquda_gammas_order} = [15, 8, 7, 1, 14, 2, 13, 4, 11, 0, 9, 3, 5, 10, 6, 12]. \quad (4)$$

The Dirac gamma matrices relevant for derivative insertions are

$$\gamma_1 \leftrightarrow \text{gamma}(1), \quad \gamma_2 \leftrightarrow \text{gamma}(2), \quad \gamma_3 \leftrightarrow \text{gamma}(4), \quad \gamma_4 \leftrightarrow \text{gamma}(8). \quad (5)$$

The chiral matrix is $\gamma_5 \leftrightarrow \text{gamma}(15)$.

The explicit representation returned by the current PyQUDA implementation is

$$\gamma_1 = \begin{pmatrix} 0 & 0 & 0 & i \\ 0 & 0 & i & 0 \\ 0 & -i & 0 & 0 \\ -i & 0 & 0 & 0 \end{pmatrix}, \quad \gamma_2 = \begin{pmatrix} 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \end{pmatrix}, \quad (6)$$

$$\gamma_3 = \begin{pmatrix} 0 & 0 & i & 0 \\ 0 & 0 & 0 & -i \\ -i & 0 & 0 & 0 \\ 0 & i & 0 & 0 \end{pmatrix}, \quad \gamma_4 = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}, \quad (7)$$

with $\gamma_5 = \text{diag}(1, 1, -1, -1)$. Every connected and disconnected EMT file stores these matrices in `gamma_matrices`; the shared implementation source is `fermion_bilinear_basis.py`.

These are raw PyQUDA bit-mask matrices. The production files store `physical_from_pyquda` so that analysis can use a uniform physical definition. With $\gamma_5 = \gamma_1\gamma_2\gamma_3\gamma_4$, raw X5 and Z5 already equal $\gamma_\mu\gamma_5$, whereas raw Y5 and T5 require a minus sign. Raw tensor masks equal $[\gamma_\mu, \gamma_\nu]/2$; multiply by i for the Hermitian $i[\gamma_\mu, \gamma_\nu]/2$ convention.

1.3 Charge Conjugation Matrix

The charge conjugation matrix in the Euclidean DeGrand–Rossi basis is

$$C = i\gamma_2\gamma_4 = i \text{ gamma}(2) @ \text{ gamma}(8). \quad (8)$$

It satisfies the standard relations

$$C\gamma_\mu^T C^{-1} = -\gamma_\mu, \quad C^T = -C, \quad C^\dagger = C^{-1} = -C. \quad (9)$$

1.4 Propagator Data Layout

A lattice quark propagator in PyQUDA is stored with the index structure

$$S(x, x_0) \longleftrightarrow [\mathbf{w}, \mathbf{t}, \mathbf{z}, \mathbf{y}, \mathbf{x_cb}, \text{spin_sink}, \text{spin_src}, \text{color_sink}, \text{color_src}], \quad (10)$$

where $\mathbf{w}$ is the checkerboard/parity index, $(\mathbf{t}, \mathbf{z}, \mathbf{y}, \mathbf{x_cb})$ are the local spacetime coordinates, and the remaining four indices carry the spin and color degrees of freedom of the two ends of the propagator. In einsum notation we label this layout as

$$S_{a,d}^{i,k}(x, x_0) \quad \text{with} \quad i = \text{spin_sink}, k = \text{spin_src}, a = \text{color_sink}, d = \text{color_src}. \quad (11)$$

Greek letters from the middle of the alphabet ($\alpha, \beta, \gamma, \dots$) denote spin indices and Latin letters ($a, b, c, \dots$) denote color indices when we write mathematical formulas. In the einsum code listings, spin indices are i, j, k, l, m, n and color indices are a, b, c, d, e, f .

1.5 Momentum Phases

Momentum projection uses the `MomentumPhase` class from PyQUDA. For a momentum $k = (k_1, k_2, k_3, 0)$ and source position x_0 , the phase at position r is

$$\Phi_k(r - x_0) = \exp(-i k \cdot (r - x_0)), \quad (12)$$

where the sign convention follows the `MomentumPhase` implementation and the dot product uses the spatial components only.

Part I

Proton Interpolating Field and Two-Point Function

2 Proton Interpolating Field

The proton is interpolated by a color-singlet three-quark operator with spin- $\frac{1}{2}$. The standard choice used in the code is the scalar-diquark interpolator. Expressed in terms of up and down quark fields, the proton interpolating field at position y is

$$\chi_\alpha(y) = \varepsilon_{abc} \left[u_a^T(y) C \Gamma_{\text{interp}} d_b(y) \right] u_{c,\alpha}(y), \quad (13)$$

where:

- $\alpha = 1, \dots, 4$ is the free Dirac index of the proton;
- $a, b, c = 1, 2, 3$ are fundamental color indices;

- ε_{abc} is the totally antisymmetric Levi-Civita tensor with $\varepsilon_{123} = +1$;
- $C = i\gamma_2\gamma_4$ is the charge conjugation matrix;
- Γ_{interp} selects the spin structure of the diquark channel;
- $u(x), d(x)$ are the up- and down-quark Dirac spinor fields.

The transpose u_a^T acts in Dirac space only; the color index a is contracted via the epsilon tensor. The bracketed term $[u^T C \Gamma_{\text{interp}} d]$ forms a Lorentz scalar (for $\Gamma_{\text{interp}} = \gamma_5$) or a pseudoscalar diquark, and the remaining up quark $u_{c,\alpha}$ carries the spin of the proton.

2.1 Supported Interpolators

The code supports three choices for Γ_{interp} , mapped by the string argument `interpolator`:

<code>interpolator</code>	Γ_{interp}	Description
"5"	$C\gamma_5$	Scalar diquark (standard)
"T5"	$C\gamma_t\gamma_5$	Temporal-axial diquark
"Z5"	$C\gamma_z\gamma_5$	Longitudinal-axial diquark

In PyQUDA code:

```
Cg5  = (1j * gamma.gamma(2) @ gamma.gamma(8)) @ gamma.gamma(15)
CgT5 = (1j * gamma.gamma(2) @ gamma.gamma(8)) @ gamma.gamma(7)
CgZ5 = (1j * gamma.gamma(2) @ gamma.gamma(8)) @ gamma.gamma(11)
```

Note that $C = i\gamma_2\gamma_4$ is built first, then right-multiplied by the appropriate gamma structure.

2.2 Epsilon Tensor Convention

The color epsilon tensor is defined as

$$\varepsilon_{abc} = \begin{cases} +1 & \text{if } (a, b, c) \text{ is an even permutation of } (0, 1, 2), \\ -1 & \text{if } (a, b, c) \text{ is an odd permutation of } (0, 1, 2), \\ 0 & \text{otherwise.} \end{cases} \quad (14)$$

In the code, ε is initialized on the host as

```
epsilon_host = xp.zeros((3,3,3), dtype=...)
for a in range(3):
    b = (a+1) % 3
    c = (a+2) % 3
    epsilon_host[a,b,c] = 1
    epsilon_host[a,c,b] = -1
```

This implements $\varepsilon_{012} = \varepsilon_{120} = \varepsilon_{201} = +1$ and $\varepsilon_{021} = \varepsilon_{102} = \varepsilon_{210} = -1$, i.e., the standard convention with cyclic permutations positive.

3 Spin and Parity Projection

The proton two- and three-point functions are projected onto definite parity and spin states by a set of spin projection matrices $\mathcal{P}$. In the rest frame of the proton, the relevant matrices constructed in the code are:

3.1 Positive Parity Projector

The positive parity projector isolates the forward-propagating (positive-energy) component of the proton two-point function:

$$\mathcal{P}_P = \frac{\gamma_0 + \gamma_4}{4} = \frac{1}{4}(\text{gamma}(0) + \text{gamma}(8)). \quad (15)$$

Here $\gamma_0 \equiv \mathbb{K}_{4 \times 4}$ and γ_4 is the temporal Dirac matrix. In Euclidean space, this projector selects states with $p_4 \approx +im_N$.

3.2 Spin Projectors

For a spin- $\frac{1}{2}$ particle at rest, the spin projection operators in the z and x directions are

$$\mathcal{P}_{S_z}^+ = \gamma_0 - i\gamma_1\gamma_2 = \text{gamma}(0) - i\text{gamma}(1) @ \text{gamma}(2), \quad (16)$$

$$\mathcal{P}_{S_z}^- = \gamma_0 + i\gamma_1\gamma_2 = \text{gamma}(0) + i\text{gamma}(1) @ \text{gamma}(2), \quad (17)$$

$$\mathcal{P}_{S_x}^+ = \gamma_0 - i\gamma_2\gamma_4 = \text{gamma}(0) - i\text{gamma}(2) @ \text{gamma}(4), \quad (18)$$

$$\mathcal{P}_{S_x}^- = \gamma_0 + i\gamma_2\gamma_4 = \text{gamma}(0) + i\text{gamma}(2) @ \text{gamma}(4). \quad (19)$$

3.3 Combined Parity-Spin Projectors

The physical proton states are selected by combining the parity and spin projectors. The `PolProjections` dictionary contains

$$\text{"PpSzp"} = \mathcal{P}_P \mathcal{P}_{S_z}^+, \quad (20)$$

$$\text{"PpSzm"} = \mathcal{P}_P \mathcal{P}_{S_z}^-, \quad (21)$$

$$\text{"PpSxp"} = \mathcal{P}_P \mathcal{P}_{S_x}^+, \quad (22)$$

$$\text{"PpSxm"} = \mathcal{P}_P \mathcal{P}_{S_x}^-, \quad (23)$$

$$\text{"PpUnpol"} = \mathcal{P}_P \quad (\text{unpolarized}). \quad (24)$$

These five projectors are labeled by the `pol_list` parameter and applied as spin matrices with indices $(i, j) = (\text{spin_sink}, \text{spin_src})$.

4 Proton Two-Point Function

4.1 Definition

The connected proton two-point function with sink bilinear Γ_{sink} and momentum projection p is

$$C_2^{\Gamma_{\text{sink}}}(p, t_{\text{sep}}) = \sum_{\mathbf{x}} \exp(-i\mathbf{p} \cdot (\mathbf{x} - \mathbf{x}_0)) \mathcal{P}_{\alpha\alpha'} \langle \chi_\alpha(\mathbf{x}, t_{\text{sep}} + t_0) \Gamma_{\text{sink}\beta\beta'} \bar{\chi}_{\beta'}(\mathbf{x}_0, t_0) \rangle, \quad (25)$$

where the source is placed at $x_0 = (\mathbf{x}_0, t_0)$ and the sink is at $(\mathbf{x}, t = t_0 + t_{\text{sep}})$. The sink gamma matrix Γ_{sink} is scanned over all 16 bilinear structures listed in Section 1.2; in the code they are indexed by the integer label $g \in \{0, \dots, 15\}$.

The $\bar{\chi}$ conjugate field is obtained via $\bar{\chi} = \chi^\dagger \gamma_4$. After inserting the three-quark interpolators (13), the two-point function contains six fermion fields (three quarks and three antiquarks) whose Wick contractions produce the baryon propagator.

4.2 Wick Contraction Result

Performing all Wick contractions of (25) yields two topologically distinct terms, corresponding to the two ways of pairing the identical up-quark fields. In the isospin-symmetric limit (identical propagators for u and d quarks), we denote the common quark propagator from x_0 to x as

$$S_{a,d}^{i,k}(x, x_0) \equiv \langle u_a^i(x) \bar{u}_d^k(x_0) \rangle = \langle d_a^i(x) \bar{d}_d^k(x_0) \rangle, \quad (26)$$

where (i, k) are `spin_sink` and `spin_src` indices and (a, d) are `color_sink` and `color_src` indices.

The two terms read:

Term 1 (direct contraction):

$$C_2^{(1)}[g, p, t] = \sum_{\mathbf{x}} \Phi_p(\mathbf{x} - \mathbf{x}_0) \varepsilon_{abc} \varepsilon_{def} (C\Gamma_{\text{interp}})_{ij} (C\Gamma_{\text{interp}})_{kl} [\Gamma_{\text{sink}g}]_{mn} S_{a,d}^{i,k}(x, x_0) S_{b,e}^{j,l}(x, x_0) S_{c,f}^{m,n}(x, x_0). \quad (27)$$

Term 2 (exchange contraction):

$$C_2^{(2)}[g, p, t] = \sum_{\mathbf{x}} \Phi_p(\mathbf{x} - \mathbf{x}_0) \varepsilon_{abc} \varepsilon_{def} (C\Gamma_{\text{interp}})_{ij} (C\Gamma_{\text{interp}})_{kl} [\Gamma_{\text{sink}g}]_{mn} S_{a,d}^{i,k}(x, x_0) S_{b,e}^{j,n}(x, x_0) S_{c,f}^{m,l}(x, x_0). \quad (28)$$

4.3 Sign Convention

The total two-point function is

$$C_2^{\Gamma_{\text{sink}}}(p, t) = -(C_2^{(1)} + C_2^{(2)}). \quad (29)$$

The overall minus sign arises from the Grassmann nature of the fermion fields and the conventions of the epsilon tensors. It is essential for obtaining the correct positive-definite spectral decomposition of the proton two-point function.

4.4 Physical Interpretation of the Two Terms

In Term 1 (27), the index structure $S_{a,d}^{i,k} S_{b,e}^{j,l} S_{c,f}^{m,n}$ corresponds to the “direct” pairing where the first up quark at the source connects to the first up quark at the sink, the down quark connects to the down quark, and the second up quark connects to the second up quark. The spin indices contract as:

- $(C\Gamma_{\text{interp}})_{ij}$ couples the spin of propagators 1 and 2 (the diquark);
- $(C\Gamma_{\text{interp}})_{kl}$ couples the spin of the conjugate diquark;
- $[\Gamma_{\text{sink}g}]_{mn}$ provides the sink spin structure.

Term 2 (28) is the exchange term where the `spin_src` indices n and l are swapped between propagators 2 and 3. This term is mandatory for fermionic statistics: the two up quarks are identical, so their propagators must be antisymmetrized. The sum $C_2^{(1)} + C_2^{(2)}$ is antisymmetric under exchange of the two up-quark propagators, as required by the Pauli principle.

5 Optimized Two-Point Contraction

The direct evaluation of (27) and (28) via a single einsum would require a 10-index tensor contraction with memory footprint $\mathcal{O}(V \times 4^6 \times 3^6)$, which is impractical. The code in `contract_2pt_TMD` splits the large contraction into a sequence of smaller two- and three-tensor operations. We detail each step below.

5.1 Term 1: Step-by-Step Decomposition

The einsum indices used in the code are:

- $\mathbf{wtzyx}$ – local spacetime coordinates (complex, lattice-wide);
- i, j, k, l, m, n – spin indices $(0, \dots, 3)$;
- a, b, c, d, e, f – color indices $(0, 1, 2)$;
- g – sink gamma index $(0, \dots, 15)$;
- p – momentum index.

The full Term 1 contraction before optimization is

$$\text{einsum}(\text{"abc, def, ij, kl, gmn, p..., ...ikad, ...jlbe, ...mncf -> gpt"}). \quad (30)$$

The code decomposes this into eight steps:

Step 1 (t1_s1): Contract ε_{abc} with the first propagator on color index a :

$$\mathbf{t1_s1} = \text{einsum}(\text{"abc, wtzyxikad -> wtzyxikbcd"}), \quad (31)$$

producing an intermediate of shape $[\mathbf{wtzyx}, i, k, b, c, d]$. The indices (b, c) remain free for later epsilon contraction; (d) is the color_src index that will contract with ε_{def} .

Step 2 (t1_s2): Contract $(C\Gamma_{\text{interp}})_{ij}$ with the second propagator on spin index j :

$$\mathbf{t1_s2} = \text{einsum}(\text{"ij, wtzyxjlbe -> wtzyxilbe"}), \quad (32)$$

producing $[\mathbf{wtzyx}, i, l, b, e]$. Spin index j is summed; i will later be contracted with $\mathbf{t1_s1}$.

Step 3 (term1_sink): Combine the two partial sink blocks by contracting spin index i and color index b :

$$\text{term1_sink} = \text{einsum}(\text{"wtzyxikbcd, wtzyxilbe -> wtzyxklcde"}), \quad (33)$$

producing $[\mathbf{wtzyx}, k, l, c, d, e]$. This object encodes the partially assembled sink-side baryon contraction.

Step 4 (term1_p3): Contract the sink gamma matrix with the third propagator on spin indices (m, n) :

$$\text{term1_p3} = \text{einsum}(\text{"gmn, wtzyxmncf -> gwtzyxcf"}), \quad (34)$$

producing $[g, \mathbf{wtzyx}, c, f]$. This step also introduces the gamma index g for scanning all 16 sink bilinears.

Step 5 (t1_f1): Contract ε_{def} with term1_sink on color indices (d, e) :

$$\mathbf{t1_f1} = \text{einsum}(\text{"def, wtzyxklcde -> wtzyxklcf"}), \quad (35)$$

producing $[\mathbf{wtzyx}, k, l, c, f]$.

Step 6 (t1_f2): Contract $(C\Gamma_{\text{interp}})_{kl}$ to close the spin structure of the diquark:

$$\mathbf{t1_f2} = \text{einsum}(\text{"kl, wtzyxklcf -> wtzyxcf"}), \quad (36)$$

producing $[\mathbf{wtzyx}, c, f]$.

Step 7 (t1_f3): Contract with the P3 block on the remaining color indices (c, f) :

$$\mathbf{t1_f3} = \text{einsum}(\text{"wtzyxcf, gwtzyxcf -> gwtzyx"}), \quad (37)$$

producing $[g, \mathbf{wtzyx}]$.

Step 8 (term1): Contract with the momentum phases to project to definite spatial momentum and sum over space:

$$\text{term1} = \text{einsum}(\text{"pwtzyx, gwtzyx -> gpt"}), \quad (38)$$

producing the final shape $[g, p, t]$.

5.2 Term 2: Step-by-Step Decomposition

The full Term 2 contraction before optimization is

$$\text{einsum}(\text{"abc, def, ij, kl, gmn, p..., ...ikad, ...jnbe, ...mlcf -> gpt"}). \quad (39)$$

Note the index differences from Term 1:

- Prop 2 has indices $(\dots, j, n, b, e)$ instead of $(\dots, j, l, b, e)$;
- Prop 3 has indices $(\dots, m, l, c, f)$ instead of $(\dots, m, n, c, f)$.

The eight optimized steps for Term 2 are:

Step 1 (t2_s1): Same as t1_s1 (eps on first propagator):

$$\text{t2_s1} = \text{einsum}(\text{"abc, wtzyxikad -> wtzyxikbcd"}). \quad (40)$$

Step 2 (t2_s2): Contract $(C\Gamma_{\text{interp}})_{ij}$ with the second propagator on spin index j (note: n remains free instead of l):

$$\text{t2_s2} = \text{einsum}(\text{"ij, wtzyxjnbe -> wtzyxinbe"}), \quad (41)$$

producing $[\text{wtzyx}, i, n, b, e]$.

Step 3 (term2_sink): Combine on indices (i, b) , keeping k, n free:

$$\text{term2_sink} = \text{einsum}(\text{"wtzyxikbcd, wtzyxinbe -> wtzyxkncde"}), \quad (42)$$

producing $[\text{wtzyx}, k, n, c, d, e]$.

Step 4 (term2_p3): Contract sink gamma with third propagator (note: n, l both free):

$$\text{term2_p3} = \text{einsum}(\text{"gmn, wtzyxnlcf -> gwtzyxnlcf"}), \quad (43)$$

producing $[g, \text{wtzyx}, n, l, c, f]$.

Step 5 (t2_f1): Contract ε_{def} with term2_sink:

$$\text{t2_f1} = \text{einsum}(\text{"def, wtzyxkncde -> wtzyxkncf"}), \quad (44)$$

producing $[\text{wtzyx}, k, n, c, f]$.

Step 6 (t2_f2): Contract t2_f1 with term2_p3 on indices (n, c, f) :

$$\text{t2_f2} = \text{einsum}(\text{"wtzyxkncf, gwtzyxnlcf -> gwtzyxkl"}), \quad (45)$$

producing $[g, \text{wtzyx}, k, l]$.

Step 7 (t2_f3): Contract $(C\Gamma_{\text{interp}})_{kl}$:

$$\text{t2_f3} = \text{einsum}(\text{"kl, gwtzyxkl -> gwtzyx"}), \quad (46)$$

producing $[g, \text{wtzyx}]$.

Step 8 (term2): Momentum projection:

$$\text{term2} = \text{einsum}(\text{"pwtzyx, gwtzyx -> gpt"}). \quad (47)$$

5.3 Memory Optimization Rationale

The key insight of the split contraction is that the full six-propagator tensor (three propagators $\times$ two spin indices $\times$ two color indices each) is never materialized. Instead:

- Steps 1–3 build the sink block using only *two* propagators at a time, with intermediate tensors of size $\mathcal{O}(V \times 4^3 \times 3^2)$.
- Step 4 processes the third propagator with the sink gamma independently.
- Steps 5–8 assemble the final result using tensors where most spin indices have already been contracted.

The largest intermediate tensor is `term1_sink` or `term2_sink` with shape `[wtzyx, 4, 4, 3, 3, 3]`, which is $V \times 4^2 \times 3^3 = 432 V$ complex numbers – manageable for typical lattice volumes.

Part II

Proton Sequential Source and Connected Three-Point Function

6 Connected Proton Three-Point Function

6.1 Definition before the Sequential Trick

The connected proton three-point function with a local insertion $O_f(x)$ on flavor $f \in \{U, D\}$ is

$$C_3^f(q, \tau; p_f, t_{\text{sep}}, \mathcal{P}) = \sum_{\mathbf{x}, \mathbf{y}} \Phi_q(x - x_0) \Phi_{p_f}(y - x_0) \mathcal{P}_{\alpha\alpha'} \times \langle \chi_\alpha(y) O_f(x) \bar{\chi}_{\alpha'}(x_0) \rangle_{\text{connected}}. \quad (48)$$

Here:

- $x_0 = (\mathbf{x}_0, t_0)$ is the source position;
- $x = (\mathbf{x}, \tau)$ is the EMT insertion point;
- $y = (\mathbf{y}, t_0 + t_{\text{sep}})$ is the fixed proton sink;
- $q = p_f - p_i$ is the momentum transfer;
- p_f is the final-state proton momentum (set by `pf`);
- t_{sep} is the source-sink separation;
- $\mathcal{P}$ is one of the five combined spin-parity projectors from Section 3.3.

The subscript “connected” means that all three valence quark lines connect the source, insertion, and sink – no disconnected (sea-quark-loop) diagrams are included at this stage.

6.2 Sequential-Source Strategy

The double sum over sink position $\mathbf{y}$ and insertion position $\mathbf{x}$ cannot be evaluated by direct lattice summation for each operator insertion, since it would require a new inversion for every $(t_{\text{sep}}, \mathbf{x})$. Instead, the fixed-sink sequential-source method is employed:

1. Build the forward (source-to-sink) propagator $S(y, x_0)$ and apply sink smearing.
2. Contract the sink-side baryon (diquark + spectator) to produce a *sequential source* $\eta_{\text{seq}}(x; p_f, t_{\text{sep}}, \mathcal{P})$.
3. Invert the Dirac operator on the sequential source: $D \Sigma = \eta_{\text{seq}}$.
4. The sequential propagator Σ carries the full sink-side information. The three-point function is then obtained by a single contraction of Σ with the forward propagator and the operator insertion at each x .

This reduces the computational cost from $\mathcal{O}(N_{t_{\text{sep}}} \times V)$ inversions to $\mathcal{O}(N_{t_{\text{sep}}} \times N_{\text{flavor}} \times N_{\text{pol}})$ inversions, where N_{pol} is the number of polarization states.

7 Proton Fixed-Sink Sequential Source

The sequential source is built by `create_bw_seq_pyquda`, which delegates the flavor-specific diquark contraction to either `up_quark_insertion_pyquda` (flavor U , insertion on an up quark) or `down_quark_insertion_pyquda` (flavor D , insertion on the down quark).

7.1 Flavor 1: Up-Quark Insertion

When the operator insertion is on an up-quark line, the proton contains two identical up quarks, which leads to four distinct contraction terms after antisymmetrization. We define the following intermediate objects built from the forward propagator S (using isospin symmetry, $S_u = S_d = S$):

$$[\Gamma^T S \Gamma]_{ab}^{ij} \equiv (\Gamma_{\text{interp}}^T)_{ik} S_{ab}^{kl} (\Gamma_{\text{interp}})_{lj}, \quad (49)$$

$$[\mathcal{P} S]_{ab}^{ij} \equiv \mathcal{P}_{ik} S_{ab}^{kj}, \quad (50)$$

$$[S \mathcal{P}]_{ab}^{ij} \equiv S_{ab}^{ik} \mathcal{P}_{kj}, \quad (51)$$

$$[\text{Tr}_{\text{spin}}(S \mathcal{P})]_{ab} \equiv S_{ab}^{kj} \mathcal{P}_{jk}. \quad (52)$$

Equation (49) is the down-quark propagator dressed with the interpolating gamma on both sides – this is the standard structure appearing in the diquark. Equations (50) and (51) are the up-quark propagator multiplied by the spin projector from the left or right. Equation (52) is the spin-space trace of $S \mathcal{P}$, leaving only color indices.

With these definitions, the **four terms** of the up-quark sequential source are:

Term R1: Both up quarks participate in the diquark; the spin projector is applied to the output:

$$R_1^{ij; cf} = \mathcal{P}^{ij} \varepsilon_{abc} \varepsilon_{def} \left[(\Gamma^T S \Gamma)_{be}^{mn} S_{ad}^{mn} \right], \quad (53)$$

where the spin indices m, n are summed (the term $[\Gamma^T S \Gamma \cdot S]$ is a Lorentz scalar in spin space). The result carries two free spin indices (i, j) from the projector and two free color indices (c, f) from the epsilon tensors.

Term R2: The spin trace of $S\mathcal{P}$ is used, and the diquark $\Gamma^T S\Gamma$ provides the spin structure:

$$R_2^{ij;cf} = \varepsilon_{abc} \varepsilon_{def} [\text{Tr}_{\text{spin}}(S\mathcal{P})]_{ad} [\Gamma^T S\Gamma]_{be}^{ji}, \quad (54)$$

where the spin indices (j, i) of $\Gamma^T S\Gamma$ become the output spin indices (i, j) .

Term R3: Spin contraction between $[\mathcal{P}S]$ and $[\Gamma^T S\Gamma]$:

$$R_3^{ij;cf} = \varepsilon_{abc} \varepsilon_{def} [\mathcal{P}S]_{ad}^{ik} [\Gamma^T S\Gamma]_{be}^{jk}, \quad (55)$$

where the repeated spin index k is summed.

Term R4: Spin contraction between $[\Gamma^T S\Gamma]$ and $[S\mathcal{P}]$:

$$R_4^{ij;cf} = \varepsilon_{abc} \varepsilon_{def} [\Gamma^T S\Gamma]_{ad}^{ki} [S\mathcal{P}]_{be}^{kj}, \quad (56)$$

where again the spin index k is summed.

The total up-quark sequential source is

$$D_{\text{total}}^{ij;cf} = -(R_1 + R_2 + R_3 + R_4). \quad (57)$$

The overall minus sign is consistent with the minus sign in the two-point function (29), as verified by the fact that the sequential propagator contracted with the forward propagator reproduces the two-point function when the insertion is the identity operator.

7.2 Flavor 2: Down-Quark Insertion

For the down-quark insertion there is only one down quark in the proton, so the antisymmetrization produces only two terms. The intermediate objects are

$$[\mathcal{P}S]^{fc} \equiv \mathcal{P}_{fk} S_c^k \quad (\text{spin_sink } f, \text{ spin_src } c), \quad (58)$$

$$[\Gamma^T S\Gamma]_{eb}^{uv} \equiv (\Gamma_{\text{interp}}^T)_{uk} S_{eb}^{kl} \Gamma_{\text{interp}lv}, \quad (59)$$

$$[\Gamma^T S]_{ec}^{uj} \equiv (\Gamma_{\text{interp}}^T)_{uk} S_{ec}^{kj}, \quad (60)$$

$$[\mathcal{P}S\Gamma]_{fb}^{jk} \equiv \mathcal{P}_{jl} S_{fb}^{lm} \Gamma_{\text{interp}mk}. \quad (61)$$

(For compactness we suppress the common spacetime and color indices in the notation above; the full forms are given in the einsum expressions below.)

The two terms are:

Term 1 (down insertion):

$$D_{1(\text{down})}^{uv;ad} = \varepsilon_{abc} \varepsilon_{def} [\mathcal{P}S]^{fc} [\Gamma^T S\Gamma]_{eb}^{uv}, \quad (62)$$

where the spin structure comes entirely from $\Gamma^T S\Gamma$, and $[\mathcal{P}S]$ is traced in all internal indices that connect to the epsilon contraction (the spin index c and color indices are summed through the epsilons).

Term 2 (down insertion):

$$D_{2(\text{down})}^{uv;ad} = \varepsilon_{abc} \varepsilon_{def} [\Gamma^T S]_{ec}^{uj} [\mathcal{P}S\Gamma]_{fb}^{jk}, \quad (63)$$

where the spin index j and color index f are summed with the epsilon contraction.

The total down-quark sequential source is

$$D_{(\text{down})}^{uv;ad} = D_2 - D_1. \quad (64)$$

7.3 From Diquark Contraction to Sequential Inversion

After the diquark contraction, the sequential source construction proceeds as follows (for each flavor and polarization):

1. **Time slicing.** The diquark-contracted object is restricted to the sink time slice $t_{\text{sink}} = (t_0 + t_{\text{sep}}) \bmod L_t$ using `sequential12`, which sets the source to zero at all other time slices.
2. **Momentum phase and γ_5 conjugation.** The sliced source is multiplied by the final-state momentum phase $\Phi_{p_f}(y - x_0)$ and conjugated with γ_5 :

$$\eta_{\text{seq},a,b}^{i,k}(y) = [\gamma_5]_{ij} \Phi_{p_f}(y - x_0) [D^\dagger]_{b,a}^{j,k}, \quad (65)$$

where D is the diquark contraction result from (57) or (64). This γ_5 -hermiticity step is required to convert the source into the correct form for the sequential inversion with the same Dirac operator used for the forward propagator.

3. **Optional boosted smearing.** When `CG_GaussSmear=True`, the sequential source is smeared using `boosted_smearing` with width w and boost vector `boost_out`. When it is false, source, two-point sink, propagator sink, and sequential-source smearing are all skipped, giving a genuine point-source/point-sink workflow.
4. **Sequential inversion.** The smeared source is inverted:

$$\Sigma = D^{-1} \eta_{\text{seq},\text{smeared}}. \quad (66)$$

5. **Final γ_5 conjugation.** The sequential propagator is conjugated once more with γ_5 to obtain the final object:

$$\Sigma_{\text{final},c,f}^{i,j}(x) = [\Sigma^\dagger]_{f,c}^{j,i}(x) [\gamma_5]_{i,j}. \quad (67)$$

In einsum notation:

$$\text{final_term} = \text{einsum}(\text{"wtzyxijfc, ik -> wtzyxjkcf"}, \text{seq_prop.conj}(), \text{G5}). \quad (68)$$

7.4 Sequential Object Shape

The final sequential object (one per polarization) has the shape

$$\boxed{\Sigma[\mathcal{P}] \longleftrightarrow [\text{w}, \text{t}, \text{z}, \text{y}, \text{x_cb}, \text{spin_a}, \text{spin_b}, \text{color_a}, \text{color_b}]}, \quad (69)$$

where:

- `spin_a`, `spin_b` are the Dirac indices at the two ends of the sequential propagator (corresponding to the insertion and sink ends);
- `color_a`, `color_b` are the corresponding color indices.

In einsum notation we label these as indices (j, i, c, f) for `(spin_a, spin_b, color_a, color_b)`. Note the index ordering: the sequential propagator’s “source” (insertion-side) indices are (i, f) and its “sink”-side indices are (j, c) .

The full list of sequential objects is gathered into an array

$$\text{seq_bw} \longleftrightarrow [\text{polarization}, \text{w}, \text{t}, \text{z}, \text{y}, \text{x_cb}, \text{spin_a}, \text{spin_b}, \text{color_a}, \text{color_b}]. \quad (70)$$

Part III

Proton EMT Contractions

8 Scalar Diagnostic: C_3^χ

The scalar diagnostic three-point function tests the sequential-source machinery by contracting the sequential propagator with the forward propagator without any derivative insertion. For flavor f ,

$$C_3^{\chi,f}(q, \tau) = \sum_{\mathbf{x}} \Phi_q(x - x_0) [\Sigma^f(x)]_{c,f}^{j,i} [S(x, x_0)]_{f,c}^{i,j}, \quad (71)$$

where the spin indices (j, i) of the sequential propagator are contracted with (i, j) of the forward propagator, and color indices (c, f) with (f, c) . This is the full spin-color trace that, for $q = 0$, reproduces the zero-momentum two-point function $C_2(t_{\text{sep}})$ up to a normalization factor.

In einsum notation:

$$\text{C3_chi} = \text{contract}(\text{"wtzyxjicf, wtzyxijfc -> wtzyx"}, \text{seq_data}, \text{prop_fw.data}). \quad (72)$$

The resulting scalar field is then projected to spatial momentum and the time dependence is extracted.

9 Connected Quark EMT Insertion

9.1 The Quark EMT Operator

In Euclidean continuum QCD, the symmetric traceless quark energy-momentum tensor is

$$T_{\mu\nu}^q(x) = \frac{1}{4} \bar{\psi}(x) \left(\gamma_\mu \overleftrightarrow{D}_\nu + \gamma_\nu \overleftrightarrow{D}_\mu \right) \psi(x), \quad (73)$$

where $\overleftrightarrow{D}_\mu = \overrightarrow{D}_\mu - \overleftarrow{D}_\mu$ is the symmetric covariant derivative. On the lattice, we implement this as a sum of two terms: the derivative acting on the forward quark line and the derivative acting on the sequential (backward) quark line.

9.2 First Term: Derivative on the Forward Line

The forward quark line runs from the source to the insertion point. Applying the covariant derivative to it gives

$$C_{3,f,\mu\nu}^{\text{first}}(q, \tau) = +\frac{1}{2} \sum_{\mathbf{x}} \Phi_q(x - x_0) [\Sigma^f(x)]_{c,f}^{j,i} [\gamma_\nu D_\mu S(x, x_0)]_{f,c}^{i,j}, \quad (74)$$

where the positive sign and the factor $1/2$ come from the decomposition of the symmetrized EMT into forward and backward derivatives.

The implementation first applies the symmetric covariant derivative to the forward propagator:

$$D_\mu S = \frac{1}{2} (D_{+\mu} - D_{-\mu}) S, \quad (75)$$

where $D_{\pm\mu}$ are the gauge-covariant forward/backward finite differences with the appropriate gauge link parallel transport. Then γ_ν is contracted on the spin_sink index:

$$\text{gamma_D_fw} = \text{contract}(\text{"ab, wtzyxbdi j -> wtzyxad i j"}, \text{gamma_nu}, \text{D_fw.data}). \quad (76)$$

Finally, the spin-color trace with the sequential propagator is formed:

$$\text{scalar_field} = 0.5 * \text{contract}(\text{"wtzyxjicf, wtzyxijfc -> wtzyx"}, \text{seq_data}, \text{gamma_D_fw}). \quad (77)$$

9.3 Second Term: Derivative on the Sequential Line

The sequential propagator runs from the sink to the insertion point (backward in time). The left-acting derivative is applied to it:

$$C_{3,f,\mu\nu}^{\text{second}}(q, \tau) = -\frac{1}{2} \sum_{\mathbf{x}} \Phi_q(x - x_0) [\overleftarrow{D}_\mu \Sigma^f(x)]_{c,f}^{j,i} [\gamma_\nu S(x, x_0)]_{f,c}^{i,j}, \quad (78)$$

where $\overleftarrow{D}_\mu$ denotes the covariant derivative acting to the left (on the sequential propagator's source index). The negative sign arises because $\overleftarrow{D}_\mu = -\overrightarrow{D}_\mu^\dagger$ in the bilinear form.

In the implementation, the left derivative is applied to the raw sequential propagator before the final proton-specific γ_5 conjugation:

1. `create_bw_seq_raw_pyquda(...)` returns only the raw sequential propagator immediately after the sequential inversion. This avoids building and storing the finalized qTMD-style `dst_seq` object in the proton EMT workflow.
2. The symmetric covariant derivative $\frac{1}{2}(D_{+\mu} - D_{-\mu})$ is applied to this raw sequential propagator.
3. The derivative result is converted to the final proton sequential layout with the same γ_5 hermiticity/index transformation used in `create_bw_seq_pyquda`:

$$\text{leftD_seq} = \text{contract}(\text{"wtzyxijfc", ik} \rightarrow \text{"wtzyxjkcf"}, D_{\text{raw_seq.conj}}, G5). \quad (79)$$

This mirrors the meson EMT construction, where the left derivative is built by applying the covariant derivative to the raw sequential propagator and only then forming the backward line through γ_5 hermiticity. The convention is also the lattice realization of the standard quark EMT operator used in proton GFF calculations, $\bar{\psi} \overleftrightarrow{D}_{\{\mu\gamma_\nu\}} \psi$. This is the same operator-level convention used, for example, in the MIT/Fermilab proton gravitational form-factor calculation by Hackett, Pefkou, and Shanahan, Phys. Rev. Lett. 132, 251904 (2024), doi:10.1103/PhysRevLett.132.251904.

9.4 Full Connected Quark EMT

The total connected quark EMT three-point function is the sum of the two terms:

$$C_{3,f,\mu\nu}^q(q, \tau) = C_{3,f,\mu\nu}^{\text{first}}(q, \tau) + C_{3,f,\mu\nu}^{\text{second}}(q, \tau). \quad (80)$$

After computing the full tensor, explicit symmetrization is enforced:

$$C_{3,f,\mu\nu}^q \rightarrow \frac{1}{2}(C_{3,f,\mu\nu}^q + C_{3,f,\nu\mu}^q), \quad C_{3,f,\nu\mu}^q = C_{3,f,\mu\nu}^q. \quad (81)$$

This guarantees $T_{\mu\nu} = T_{\nu\mu}$ at the level of the measured data. The production kernel actually saves all 16 local insertion matrices and all 16×4 unsymmetrized one-derivative insertions. It constructs each forward and left-acting derivative once, batches the 16 Gamma contractions, and derives the displayed vector EMT from the four vector channels. Therefore the extended basis adds no inversion, flow update, or derivative application.

10 Key Differences from the Meson EMT

The proton EMT contraction differs from the meson (pion) case in several important ways:

1. **No `_make_dst2` required.** In the meson code, the backward sequential line is formed by $\gamma_5 \Sigma^\dagger \gamma_5$ (the `dst2` object) before each contraction. In the proton code, the sequential object *already* has the correct spin structure built in during the diquark contraction and the final γ_5 conjugation step. Therefore Σ can be used directly without the `dst2` transformation.

2. **Direct einsum contraction.** The meson contraction is a three-way trace:

$$\text{contract}(\text{"wtzyxabij, wtzyxbcji, ca -> wtzyx"}, \text{dst2}, \text{gamma.D_fw}, \text{src.gamma}). \quad (82)$$

The proton contraction is simpler:

$$\text{contract}(\text{"wtzyxjicf, wtzyxijfc -> wtzyx"}, \text{seq_data}, \text{prop_fw_data}), \quad (83)$$

summing over spin indices (j, i) and color indices (c, f) . There is no source gamma matrix in the proton contraction because the spin projection is already absorbed into the sequential source.

3. **Covariant derivative on the raw sequential propagator.** In the meson case, the left derivative is applied to the raw sequential propagator and then converted to the backward line through `_left_covdev_dst2_from_prop`. The proton EMT code now follows the same logic: the derivative acts on the raw sequential propagator, and the result is then converted to the final proton sequential layout through γ_5 hermiticity. The already-finalized `seq_data` is not treated as an ordinary `LatticePropagator` for the left derivative.

4. **Flavor dimension.** The proton result carries a flavor index $[U, D]$ since the insertion can be on either quark line. The meson result has no flavor index (only one valence flavor).

5. **Polarization dimension.** The proton result has a polarization index from `pol_list` (five entries). The pion has no spin degree of freedom.

6. **Time separation loop.** The proton code loops over `t_separations`, building new sequential sources for each t_{sep} . The meson code has the same structure, but the proton sequential source construction involves the epsilon-tensor diquark contraction instead of a simple meson two-point contraction.

11 Gradient Flow

11.1 Flow Schedule

The gradient flow is applied identically to the meson and proton codes. The flow schedule is:

Step index	Flow action	Description
$s = 0$	None (measure unflowed)	Unflowed (bare) operators
$s = 0 \rightarrow 1$	10 sub-steps of $\varepsilon/10$ each	First flow interval subdivided
$s = 1 \rightarrow 2$	1 step of ε	Regular flow step
...	...	...
$s = N - 1 \rightarrow N$	1 step of ε	Final flow step

Thus output index `step` corresponds to flow time $t_{\text{flow}} = \text{step} \times \varepsilon$, with $t_{\text{flow}} = 0$ at `step` = 0. The subdivision of the first interval ensures that the initial condition of the flow equation is well resolved numerically.

The production implementation keeps the existing one-branch flow $[S_{\text{fw}}, S_{\text{seq}}]$, containing the 24 spin-color propagator fields. It does not batch different source positions, separations, flavors, or polarizations. After the initial full gauge and multigrid construction, the first source and

first flavor/separation sequential inversion use the already-resident original gauge. Inversions following a flowed-gauge context restore it with `thin_update_only=True`. At each flow time a single `U.f.use()` context supplies all forward/backward covariant derivatives for the two quark legs. These are performance changes only and leave the operator and HDF5 schema unchanged.

The 24-field branch flow remains the dominant memory constraint. In the `cfg1050` light-quark 64^4 , 16-rank smoke (local volume 32^4), the same flow allocation exhausted an 80-GB A100. This requires a finer MPI decomposition or a future connected-memory optimization for that setup; it is not addressed by thin gauge restore or by the shared derivative context.

11.2 Flowing Fermion and Gauge Fields Together

At each flow step, both the forward propagator and the sequential propagator are flowed simultaneously on the same flowed gauge background. This is accomplished by packing both propagators into a single `MultiLatticeFermion` and calling `gradientFlow` once:

$$(S_{\text{flowed}}, \Sigma_{\text{flowed}}) = \text{gradientFlow}([S, \Sigma], \text{flow_type}, N_{\text{steps}}, \Delta t). \quad (84)$$

The gauge field used for the fermion flow is set to anti-periodic temporal boundary conditions via `U.f.setAntiPeriodicT()`. A copy of the unflowed gauge field is used (`U_f = U.copy()`) so that the original gauge field is preserved for subsequent measurements.

11.3 Physical Significance

Gradient flow smooths ultraviolet fluctuations at a physical radius approximately $\sqrt{8t_{\text{flow}}}$. The EMT observables measured at finite flow time are flowed composite operators. Their flow-time dependence is analyzed using the small-flow-time expansion to extract renormalized matrix elements. This analysis is performed *outside* the contraction kernel; the kernel only produces the flowed data at each `step`.

The fermion-flow kernel is four dimensional, so the quoted radius includes the Euclidean time direction and is a diffusion scale rather than a strict support boundary. This matters for stochastic disconnected loops, as summarized in Sec. 14.2.

12 Code Structure Overview

The proton EMT measurement code is organized in the class `ProtonQuarkEMT`, which inherits from `EMTDisconnectedQuark1pt`. The inheritance provides:

- The stochastic quark one-point loop measurement (`flowed_fermionic_1pt`);
- The covariant derivative utilities (`_covdev_sym_prop`, `_make_dst2`, etc.);
- The propagator flow utilities (`_advance_floweds_props`);
- Gamma matrix helpers (`_dirac_gammas_for`, `_gamma5_for`, `_get_interpolator_gamma_for`).

The key methods of `ProtonQuarkEMT` are:

`connected_3pt(gauge, invPara, source_jobs, interpolator, ...):`

- Builds the Dirac solver and the forward point-source propagator (via `_make_source_prop`).
- Contracts the proton two-point function for normalization (via `contract_proton_2pt`, which reuses `proton_TMD.contract_2pt_TMD`).
- Loops over t_{sep} and flavor, building sequential sources via `create_bw_seq_pyquda`.

- Loops over polarization and flow steps, calling `get_C3_primitive_bilinears_proton` once at each step and deriving both C_3^X and $C_3^{T_{\mu\nu}}$.
- Advances the flow between steps.
- Saves results to HDF5.

`get_C3_primitive_bilinears_proton(U_f, prop_fw, raw_seq_prop, phases_3pt, t0):`

- Computes all 16 local and 16×4 derivative primitive channels.
- Derives the scalar identity channel and symmetric vector EMT without repeating covariant derivatives.

`_seq_to_prop(latt_info, seq_data):`

- Creates a `LatticePropagator` from the sequential data array for use with propagator-level methods.

13 Output Structure

13.1 HDF5 Output for Connected Proton Three-Point

The proton connected EMT three-point files save only three-point observables and their momentum-transfer metadata. The matching two-point functions are written separately under the `EMTproton2pt` output tree.

Dataset	Shape	Description
C3_chi	[Nflavor, Npol, Nsteps+1, Nq, t_sep+2]	Scalar diagnostic three-point function
C3_Tmunu	[Nflavor, Npol, Nsteps+1, Nq, 4, 4, t_sep+2]	Quark EMT three-point function
C3_local_bilinear	[Nflavor, Npol, 16, Nq, Nsteps+1, t_sep+2]	Complete local Dirac basis
C3_derivative_bilinear	[Nflavor, Npol, 16, 4, Nq, Nsteps+1, t_sep+2]	Unsymmetrized one-derivative basis
momentum_transfer_list	[Nq, 4]	List of momentum transfer vectors

The raw derivative primitive reproduces the stored tensor with the following axis transformation:

```
with h5py.File(path, "r") as h5:
    labels = [x.decode() for x in h5["gamma_list"][...]]
    vector = [labels.index(x) for x in ("X", "Y", "Z", "T")]
    D = h5["C3_derivative_bilinear"][...]
    # D: [flavor,polarization,gamma,derivative,q,flow,t]
    B = np.take(D, vector, axis=2)
    T = 0.5 * (B + np.swapaxes(B, 2, 3))
    T_file = T.transpose(0, 1, 5, 4, 2, 3, 6)
    np.testing.assert_allclose(T_file, h5["C3_Tmunu"][...])
```

These are connected correlators, so no additional `volume_norm` division is made. The complete physical-basis and tensor-current examples are in `docs/EMT_gamma_and_raw_bilinears.md`.

The four main connected arrays have the shape-independent logical payload ratio

$$\text{C3_derivative}:\text{C3_local}:\text{C3_Tmunu}:\text{C3_chi} = 64 : 16 : 16 : 1, \quad (85)$$

or 65.98%, 16.49%, 16.49%, and 1.03% for a large file. In the measured S8T8 one-polarization, nine-momentum, two-flow-time `tsep=4` file, HDF5 metadata changes the complete-file shares to 61.78%, 15.44%, 15.44%, and 0.97%; basis datasets use 2.29% and metadata/overhead uses 4.08%.

The dimensions are:

- $N_{\text{flavor}} = 2$: index 0 = up-quark insertion, index 1 = down-quark insertion;
- $N_{\text{pol}} = \text{number of polarization states from } \text{pol_list} \text{ (typically 5)}$;
- one file represents one source–sink separation; `t_sep` is a scalar attribute and there is no singleton separation axis;
- $N_{\text{steps}} + 1 = \text{number of flow times (including unflowed)}$;
- $N_q = \text{number of momentum transfers}$;
- $N_t = \text{temporal lattice extent}$.

The attributes store configuration, Dirac parameters, the actual multigrid hierarchy, gauge preprocessing, temporal boundary, source position, source/sink interpolators, separate source/sink/sequential smearing flags and boosts, p_f , q , two-point momenta, flow schedule, scalar `t_sep`, and explicit dataset-axis labels. They also record convention metadata:

- `operator_normalization = unringed_flow_bilinear`;
- `ringed_normalization_applied = False`;
- `flow_times = flow_epsilon * [0, ..., Nsteps]`;
- `quark_flow_scope = inserted_operator_quark_legs_only`;
- `nucleon_interpolator_flow_bilinear = False`;
- `ringed_factor_source = analysis_from_quark_1pt_kinetic`.

These attributes do not change any dataset or contraction. They only make explicit that `C3_Tmunu` is an unringed flowed bilinear and must not be treated as already ringed in downstream analysis.

13.2 HDF5 Output for Disconnected Pieces

The disconnected (quark loop and gluon) one-point functions are stored separately by the inherited methods:

- Quark 1pt: the EMT shard finalizer saves per-noise-vector primitive data (`raw/local_bilinear_pervec`, `raw/derivative_bilinear_pervec`, and `raw/flowed_noise_norm_pervec`) and noise-averaged primitives plus the derived `avg/Tmunu/T11–T44`.
- Gluon 1pt: `save_emt_gluon_1pt_hdf5` – saves momentum-projected gluonic EMT building blocks (`Tmunu/T11–T44`).

13.3 File Organization

The EMT output files are organized in subdirectories:

- EMTproton2pt/ – proton two-point functions;
- EMTproton3pt/ – proton connected three-point functions;
- EMTc/ – stochastic quark one-point loops (shared with meson);
- EMTg/<lat>.EMTg.<cfg>.<ama>.<sm>.h5 – source-independent flowed gluon one-point functions shared by all hadrons.

The canonical EMTc loop name is source independent, <lat>.EMTc.<cfg>.<ama>.<sm>.h5. It stores all absolute insertion times and is reused for every proton two-point source time on that configuration. Connected proton three-point file names append the sink kinematics as PX<px>PY<py>PZ<pz>dt<tsep>. A zero-momentum sink with one source–sink separation at nine carries the suffix PX0PY0PZ0dt9.h5; separations six, nine, and twelve are written as three files with suffixes dt6.h5, dt9.h5, and dt12.h5. Each file stores scalar `t_sep` metadata and has no singleton source–sink-separation dataset axis. This prevents runs with different final momenta or source–sink separations from overwriting one another and follows the nucleon TMD output convention.

14 Disconnected Analysis

14.1 Combined Proton EMT

The full proton matrix element of the EMT receives contributions from both connected and disconnected diagrams:

$$\langle p' | T_{\mu\nu} | p \rangle = \langle p' | T_{\mu\nu} | p \rangle_{\text{conn}} + \langle p' | T_{\mu\nu} | p \rangle_{\text{disc}}. \quad (86)$$

The connected contribution is computed by `ProtonQuarkEMT.connected_3pt` as described in Sections 6.1–9.4.

14.2 Disconnected Quark Contribution

The disconnected quark contribution is evaluated using stochastic estimators. For each noise vector $\xi^{(r)}$, one solves $D\eta^{(r)} = \xi^{(r)}$ and computes the one-point function (spatially Fourier-transformed loop):

$$L_{q,\mu\nu}^{(r)}(q, t) = -\frac{1}{2} \sum_{\mathbf{x}} \Phi_q(\mathbf{x}) \xi^{(r)\dagger}(x) \gamma_\nu [D_{+\mu} - D_{-\mu}] \eta^{(r)}(x). \quad (87)$$

At finite flow time the code evolves both stochastic fields with the same four-dimensional fermion-flow kernel:

$$\xi_f = K(t_f)\xi, \quad \eta_f = K(t_f)D^{-1}\xi. \quad (88)$$

Writing P_τ for the absolute insertion-time projector, the estimator and its expectation are

$$\widehat{L}(\tau, t_f) = \xi^\dagger K^\dagger P_\tau \Gamma K D^{-1} \xi, \quad (89)$$

$$\mathbb{E}_\xi[\widehat{L}(\tau, t_f)] = \text{Tr} \left[P_\tau \Gamma K(t_f) D^{-1} K^\dagger(t_f) \right]. \quad (90)$$

Thus the physical loop remains a spatial trace at fixed τ , but the initial stochastic vector must span all Euclidean times because $K(t_f)$ spreads in the temporal direction. A single initial-time projector does not reproduce this finite-flow trace. Complete time dilution would be unbiased

after summing all projectors, but the current workflow instead uses full-volume counter-based Z_4 sources and retains every insertion time. This also avoids the repeated-local-source failure that occurs when equal-shaped MPI ranks reset an ordinary backend RNG to the same seed; rank-dependent seeds would still make the source depend on the decomposition. Spatial HP and isotropic 4D HP both preserve this full-volume support; only their probing signs differ. A detailed derivation is provided in `docs/EMT_disconnected_1pt/EMT_disconnected_1pt.tex`.

The disconnected three-point function is then obtained from the correlation with the proton two-point function:

$$C_{3,\mu\nu}^{\text{disc},q}(q, \tau) = \left\langle C_2(t_{\text{sep}}) \frac{1}{N_v} \sum_{r=1}^{N_v} L_{q,\mu\nu}^{(r)}(q, \tau) \right\rangle - \langle C_2(t_{\text{sep}}) \rangle \left\langle \frac{1}{N_v} \sum_{r=1}^{N_v} L_{q,\mu\nu}^{(r)}(q, \tau) \right\rangle. \quad (91)$$

The subtraction of the product of averages removes the vacuum expectation value of the loop.

14.3 Disconnected Gluon Contribution

The flowed gluonic EMT building block is built from the clover field strength:

$$F_{\mu\nu}(x) = -\frac{i}{8} \left(Q_{\mu\nu}(x) - Q_{\mu\nu}^\dagger(x) - \frac{1}{N_c} \text{Tr} [Q_{\mu\nu}(x) - Q_{\mu\nu}^\dagger(x)] \right), \quad (92)$$

where $Q_{\mu\nu}$ is the average of the four plaquettes in the (μ, ν) plane (clover combination). The gluonic EMT one-point building block is

$$L_{g,\mu\nu}(q, t) = \frac{2}{V_3} \sum_{\mathbf{x}} \Phi_q(\mathbf{x}) \sum_{\rho \neq \mu, \nu} \text{Tr}_c [F_{\mu\rho}(x) F_{\nu\rho}(x)]. \quad (93)$$

This quantity is measured on the flowed gauge field at each flow time. Its disconnected contribution to the proton matrix element is

$$C_{3,\mu\nu}^{\text{disc},g}(q, \tau) = \left\langle C_2(t_{\text{sep}}) L_{g,\mu\nu}(q, \tau) \right\rangle - \langle C_2(t_{\text{sep}}) \rangle \langle L_{g,\mu\nu}(q, \tau) \rangle. \quad (94)$$

14.4 Ringed-Fermion Normalization

The connected and disconnected quark EMT outputs in this repository are *unringed* flowed bilinears. They do not contain the ringed-fermion normalization factor. The ringed factor should be computed once in the analysis stage from the quark one-point kinetic expectation value, then applied consistently to both connected and disconnected quark EMT observables at the same flow time.

The conventional ringed variables of Makino–Suzuki are normalized by

$$Z_\chi^{\text{ring}}(t) = \frac{-2 \dim(R) N_f}{(4\pi)^2 t^2 \langle \bar{\chi}(t, x) \overleftrightarrow{D} \chi(t, x) \rangle}, \quad (95)$$

where $\dim(R) = N_c = 3$ for fundamental QCD and N_f is the flavor normalization convention used in the kinetic expectation value. Since a quark EMT bilinear contains one χ and one $\bar{\chi}$, the bilinear receives one factor $Z_\chi^{\text{ring}}(t)$, not its square.

At zero momentum, the diagonal quark EMT trace provides the kinetic building block:

$$\sum_{\mu} L_{q,\mu\mu}(0, t) = -\frac{1}{2} \sum_{\mathbf{x}} \langle \bar{\chi}(t, \mathbf{x}) \overleftrightarrow{D} \chi(t, \mathbf{x}) \rangle, \quad (96)$$

up to the lattice derivative convention and the spatial-volume normalization documented in the one-point file. In the current HDF5 convention,

$$K_{\text{code}}(t) = -2 \left\langle \sum_{\mu=1}^4 L_{q,\mu\mu}^{\text{avg}}(q=0, t, \tau) \right\rangle_{\tau, \text{cfg}}, \quad (97)$$

where the diagonal sum is reconstructed from `avg/Tmunu/T11`, `T22`, `T33`, and `T44`. The quark `lpt` run now performs this algebraic extraction from the same stochastic EMT tensor and writes `derived/ringed/kinetic_pervec` and `derived/ringed/kinetic_spacetime` into the same EMTc. This adds no inversion, flow update, derivative contraction, or gather.

The embedded group is intentionally kinetic-only and contains no configuration-local ringed factors. Average `derived/ringed/kinetic_spacetime` over gauge configurations first, then evaluate the nonlinear factor; averaging the inverses formed on separate configurations is incorrect. The unflowed step $t = 0$ must not be used for ringed normalization because the formula contains t^{-2} . The standalone ringed-normalization workflow remains available for dedicated kinetic-only high-statistics runs. Its independent kinetic contraction inherits the EMT base/HP-part runner and fingerprinted base-level sample-log model, without constructing the full EMT primitive basis. The production quark one-point wrapper uses base-oriented HP-interval shards; an explicit streaming finalizer validates complete base coverage before publishing one canonical EMTc with embedded kinetic data.

For isospin-degenerate light u/d measurements, the same light-quark ringed factor should multiply the connected U and D quark EMT observables. If a future workflow uses different quark masses, the analysis must either compute flavor-specific factors from flavor-specific one-point files or explicitly form the chosen flavor-average kinetic normalization before applying it.

Part IV

Appendices

A Summary of Einsum Index Conventions

For quick reference, the index labels used throughout the code and this note are:

Index	Meaning
<code>w</code>	Checkerboard/parity index
<code>t, z, y, x</code>	Local spacetime coordinates on the current MPI rank
<code>x_cb</code>	Checkerboarded spatial coordinate (in even-odd preconditioned data)
<code>i, j, k, l, m, n</code>	Spin indices (range 0–3)
<code>a, b, c, d, e, f</code>	Color indices (range 0–2 for SU(3))
<code>g</code>	Sink gamma matrix index (range 0–15)
<code>p</code>	Momentum index
<code>q</code>	Momentum transfer index
<code>pol</code>	Polarization index

Propagator layout: `[w, t, z, y, x_cb, spin_sink, spin_src, color_sink, color_src]`.

Sequential data layout: `[w, t, z, y, x_cb, spin_a, spin_b, color_a, color_b]`.

B Gamma Matrix Reference

The 16 bilinear operators and their PyQUDA gamma identifiers:

Name	Description	PyQUDA ID
"I"	Scalar (identity)	gamma(0)
"X"	γ_1	gamma(1)
"Y"	γ_2	gamma(2)
"SXY"	$\sigma_{12} = [\gamma_1, \gamma_2]/2$	gamma(3)
"Z"	γ_3	gamma(4)
"SXZ"	σ_{13}	gamma(5)
"SYZ"	σ_{23}	gamma(6)
"T5"	raw $-\gamma_4\gamma_5$	gamma(7)
"T"	γ_4	gamma(8)
"SXT"	σ_{14}	gamma(9)
"SYT"	σ_{24}	gamma(10)
"Z5"	$\gamma_3\gamma_5$	gamma(11)
"SZT"	σ_{34}	gamma(12)
"Y5"	raw $-\gamma_2\gamma_5$	gamma(13)
"X5"	$\gamma_1\gamma_5$	gamma(14)
"5"	γ_5	gamma(15)

The Dirac gamma matrices used for EMT derivative insertions:

$$\gamma_1 \leftrightarrow \text{gamma}(1) \quad (D\text{-}\gamma\text{-ID} = 1), \quad (98)$$

$$\gamma_2 \leftrightarrow \text{gamma}(2) \quad (D\text{-}\gamma\text{-ID} = 2), \quad (99)$$

$$\gamma_3 \leftrightarrow \text{gamma}(4) \quad (D\text{-}\gamma\text{-ID} = 4), \quad (100)$$

$$\gamma_4 \leftrightarrow \text{gamma}(8) \quad (D\text{-}\gamma\text{-ID} = 8). \quad (101)$$

C Complete Two-Point Contraction Einsum Map

For readers who want to verify the code against the formulas, here is the complete mapping of the two-point function contraction steps.

Term 1:

$$\text{Math: } \varepsilon_{abc} \varepsilon_{def} (CT_{\text{interp}})_{ij} (CT_{\text{interp}})_{kl} [\Gamma_{\text{sink}g}]_{mn} S_{ad}^{ik} S_{be}^{jl} S_{cf}^{mn} \longrightarrow [g, p, t]$$

(102)

$$\begin{aligned} \text{Code: } & \text{t1_s1} = \text{einsum}(\text{"abc, wtzyxikad} \rightarrow \text{wtzyxikbcd"}) \\ & \text{t1_s2} = \text{einsum}(\text{"ij, wtzyxjlbe} \rightarrow \text{wtzyxilbe"}) \\ & \text{term1_sink} = \text{einsum}(\text{"wtzyxikbcd, wtzyxilbe} \rightarrow \text{wtzyxklcde"}) \\ & \text{term1_p3} = \text{einsum}(\text{"gm, wtzyxmncf} \rightarrow \text{gwtzyxcf"}) \\ & \text{t1_f1} = \text{einsum}(\text{"def, wtzyxklcde} \rightarrow \text{wtzyxklcf"}) \\ & \text{t1_f2} = \text{einsum}(\text{"kl, wtzyxklcf} \rightarrow \text{wtzyxcf"}) \\ & \text{t1_f3} = \text{einsum}(\text{"wtzyxcf, gwtzyxcf} \rightarrow \text{gwtzyx"}) \\ & \text{term1} = \text{einsum}(\text{"pwtzyx, gwtzyx} \rightarrow \text{gpt"}) \end{aligned} \quad (103)$$

Term 2:

$$\text{Math: } \varepsilon_{abc} \varepsilon_{def} (CT_{\text{interp}})_{ij} (CT_{\text{interp}})_{kl} [\Gamma_{\text{sink}g}]_{mn} S_{ad}^{ik} S_{be}^{jn} S_{cf}^{ml} \longrightarrow [g, p, t]$$

(104)

$$\begin{aligned} \text{Code: } & \text{t2_s1} = \text{einsum}(\text{"abc, wtzyxikad} \rightarrow \text{wtzyxikbcd"}) \\ & \text{t2_s2} = \text{einsum}(\text{"ij, wtzyxjnbe} \rightarrow \text{wtzyxinbe"}) \\ & \text{term2_sink} = \text{einsum}(\text{"wtzyxikbcd, wtzyxinbe} \rightarrow \text{wtzyxknede"}) \\ & \text{term2_p3} = \text{einsum}(\text{"gm, wtzyxnlcf} \rightarrow \text{gwtzyxnlcf"}) \\ & \text{t2_f1} = \text{einsum}(\text{"def, wtzyxknede} \rightarrow \text{wtzyxknecf"}) \\ & \text{t2_f2} = \text{einsum}(\text{"wtzyxknecf, gwtzyxnlcf} \rightarrow \text{gwtzyxkl"}) \\ & \text{t2_f3} = \text{einsum}(\text{"kl, gwtzyxkl} \rightarrow \text{gwtzyx"}) \\ & \text{term2} = \text{einsum}(\text{"pwtzyx, gwtzyx} \rightarrow \text{gpt"}) \end{aligned} \quad (105)$$

Final result:

$$\text{corr} = - \text{term1} - \text{term2}. \quad (106)$$

D List of Parameters

The `parameters` dictionary expected by `ProtonQuarkEMT` and its parent classes:

Key	Description
<code>qext</code>	List of momentum transfer vectors $q = (q_x, q_y, q_z, 0)$
<code>pf</code>	Final-state proton momentum p_f
<code>p_2pt</code>	List of momenta for the two-point function
<code>CG_GaussSmear</code>	Boolean: enable gauge-covariant Gaussian smearing
<code>pos_boost</code>	Boost vector for forward source/sink smearing
<code>neg_boost</code>	Boost vector for backward source/sink smearing
<code>boost_in</code>	Boost vector for forward propagator (proton source)
<code>boost_out</code>	Boost vector for sequential propagator (proton sink)
<code>width</code>	Gaussian smearing width
<code>pol</code>	List of polarization labels (e.g., <code>["PpUnpol"]</code>)
<code>t_insert</code>	Maximum time separation (for TMD compatibility)
<code>save_propagators</code>	Boolean: save intermediate propagators
<code>flow_type</code>	Flow action type: "wilson" or "symanzik"
<code>flow_epsilon</code>	Flow step size ε
<code>flow_steps</code>	Number of flow steps N

E Sign Convention Summary

For quick reference, here are all the sign conventions used in the proton EMT code:

1. **Two-point function:** $C_2 = -(C_2^{(1)} + C_2^{(2)})$. The minus sign is required for a positive-definite spectral decomposition of the proton correlator.
2. **Sequential source (up insertion):** $D_{\text{total}} = -(R_1 + R_2 + R_3 + R_4)$. The minus sign is consistent with the two-point function sign.

3. **Sequential source (down insertion):** $D_{(\text{down})} = D_2 - D_1$. The relative minus sign between the two terms is the standard antisymmetrization of the single down quark in the baryon.
4. **Quark EMT first term:** $+\frac{1}{2}$ from the decomposition $\overleftrightarrow{D} = \overrightarrow{D} - \overleftarrow{D}$.
5. **Quark EMT second term:** $-\frac{1}{2}$, because the left derivative contributes with opposite sign in the bilinear form.
6. **Disconnected quark loop:** $-\frac{1}{2}$ overall factor, matching the connected convention. The fermion loop has an additional minus sign from the closed fermion loop.
7. **Gluon EMT building block:** Factor $+2/V_3$ from the original code convention. The clover field strength includes an extra factor of $-i$ to match the Euclidean convention.

F Validation and Regression Evidence

This section records the implementation-level validation that has been performed for the connected proton EMT measurement. These tests are not a replacement for physics production checks such as continuum normalization, renormalization, operator mixing, disconnected contributions, or final GFF extraction. They are designed to catch common implementation errors in the connected sequential-source path: gauge covariance violations, wrong left-derivative placement, phase convention mistakes, missing up-quark Wick contractions, and accidental changes to the raw-only sequential builder.

F.1 Validated Test Setup

The most complete validation suite was run on Aurora with the S8T32 test gauge

```
software_gradientflow/Pyquda_Measurement/test_gauge/
S8T32_wilson_b6.cg.1e-08.0
```

using the PyQUDA develop environment

```
software_gradientflow/activate-pyquda-develop.sh
```

with `backend="dnpn"`, `backend_target="sycl"`, two MPI ranks, and `mpi_geometry=1.1.1.2`. Parallel HDF5 was verified with `h5py.get_config().mpi == True`. Quark source/sink smearing was disabled for the gauge-covariance tests so that the tests isolate the EMT/sequential contractions rather than the non-gauge-covariant boosted-smearing path:

```
EMT_PROTON_GAUSS_SMEAR=0
bash run_proton_quark_3pt.sh --config_num 0 --mg-block none
```

HYP gauge preprocessing was kept enabled, matching the normal workflow.

The suite output is archived under

```
/lus/flare/projects/StructNGB/xgao/run/test_gauge/EMT_proton/
validation_suite_20260603_141831
```

with logs, HDF5 outputs, temporary diagnostic drivers, and JSON summaries. A companion Markdown summary is kept in `docs/proton_EMT/validation_summary.md`.

F.2 Raw-Only Sequential Builder Regression

The proton EMT path uses a raw-only sequential builder: `create_bw_seq_raw_pyquda`. The finalized proton sequential object is then produced from the raw sequential propagator by the same γ_5 -hermiticity finalization used by the original qTMD-style builder. The regression test compared

$$\text{create_bw_seq_pyquda}(\dots) \quad \text{against} \quad \text{_final_seq_from_raw_prop}(\text{create_bw_seq_raw_pyquda}(\dots)). \quad (107)$$

The measured difference was

$$\max |\Delta| = 0, \quad \frac{\max |\Delta|}{\max |C|} = 0, \quad (108)$$

confirming that the memory-saving raw-only path does not change the finalized sequential object.

F.3 Gauge Covariance of the Connected Three-Point Function

Gauge covariance was tested by comparing observables from the original gauge field, a constant global SU(3) transformed field, and a deterministic local SU(3) transformed field. The tested gauge-invariant outputs were the two-point function `C2`, the scalar/proton sequential contraction `C3_chi`, and the EMT insertion `C3_Tmunu`.

For `FLOW_STEPS=0`, `QMAX=0`, and `T_SEPS=2`, the local transform relative differences were

$$\delta_{\text{rel}}(C_2) = 1.1443 \times 10^{-10}, \quad (109)$$

$$\delta_{\text{rel}}(C_{3,\chi}) = 5.0029 \times 10^{-11}, \quad (110)$$

$$\delta_{\text{rel}}(C_{3,T_{\mu\nu}}) = 9.2440 \times 10^{-12}. \quad (111)$$

The EMT tensor was explicitly symmetric to roundoff in the stored output:

$$\max |T_{\mu\nu} - T_{\nu\mu}| = 0. \quad (112)$$

This is the key regression for the left-acting derivative fix: before applying the left derivative to the raw sequential propagator, the point-source local gauge transform test isolated an $\mathcal{O}(1)$ failure in `C3_Tmunu`; after the fix, the difference is at solver/reduction roundoff level.

F.4 Gradient-Flow Covariance

The same baseline/global/local gauge-transform test was repeated with `FLOW_STEPS=1`. The expected output shapes were observed:

$$\text{C3_chi} : [2, 1, 1, 2, 1, 32], \quad (113)$$

$$\text{C3_Tmunu} : [2, 1, 1, 2, 1, 4, 4, 32]. \quad (114)$$

The flow step changed the correlators nontrivially:

$$\max |C_{3,\chi}^{(1)} - C_{3,\chi}^{(0)}| = 3.7547 \times 10^{-5}, \quad (115)$$

$$\max |C_{3,T_{\mu\nu}}^{(1)} - C_{3,T_{\mu\nu}}^{(0)}| = 7.5175 \times 10^{-5}. \quad (116)$$

Across the full flow-step axis, the local transform relative difference for `C3_Tmunu` remained

$$\delta_{\text{rel}}(C_{3,T_{\mu\nu}}) = 9.2440 \times 10^{-12}. \quad (117)$$

F.5 Momentum Phase Checks

The QMAX=1 smoke test produced 27 unique momenta with temporal component zero, covering the full cube $[-1, 0, 1]^3$. The $q = 0$ slice of the QMAX=1 output matched the independent QMAX=0 baseline exactly:

$$\max |\Delta C_{3,\chi}(q = 0)| = 0, \quad \max |\Delta C_{3,T_{\mu\nu}}(q = 0)| = 0. \quad (118)$$

Additional MomentumPhase checks verified

$$\Phi_{-q}(x) = \Phi_q(x)^*, \quad (119)$$

the corresponding conjugation relation for a real deterministic test field, and self-consistency of the shifted-delta phase ratio; all three checks had zero measured max difference in the test.

F.6 Up-Quark Insertion Term Sanity

The up insertion source was decomposed into the four Wick-contraction terms R1, R2, R3, and R4. The optimized source in the production code was checked against the explicit decomposition:

$$D_{\text{up}} = -(R_1 + R_2 + R_3 + R_4), \quad (120)$$

with zero measured source-level difference. After time slicing, momentum projection, identity smearing, and linear sequential inversion, the raw full U sequential propagator agreed with the sum of the four term-by-term raw sequential propagators with

$$\max |\Delta| = 1.1375 \times 10^{-15}, \quad \delta_{\text{rel}} = 6.4103 \times 10^{-12}. \quad (121)$$

The U and D raw sequential objects were finite, nonzero, and not accidentally identical:

$$\delta_{\text{rel}}(U_{\text{raw}}, D_{\text{raw}}) = 0.4890. \quad (122)$$

This confirms that the two identical-up Wick-contraction channels are present in the implemented U insertion and that the optimized source respects the explicit four-term algebra.